VITALSCOPE

Nextion Control Station

Technical Documentation v2

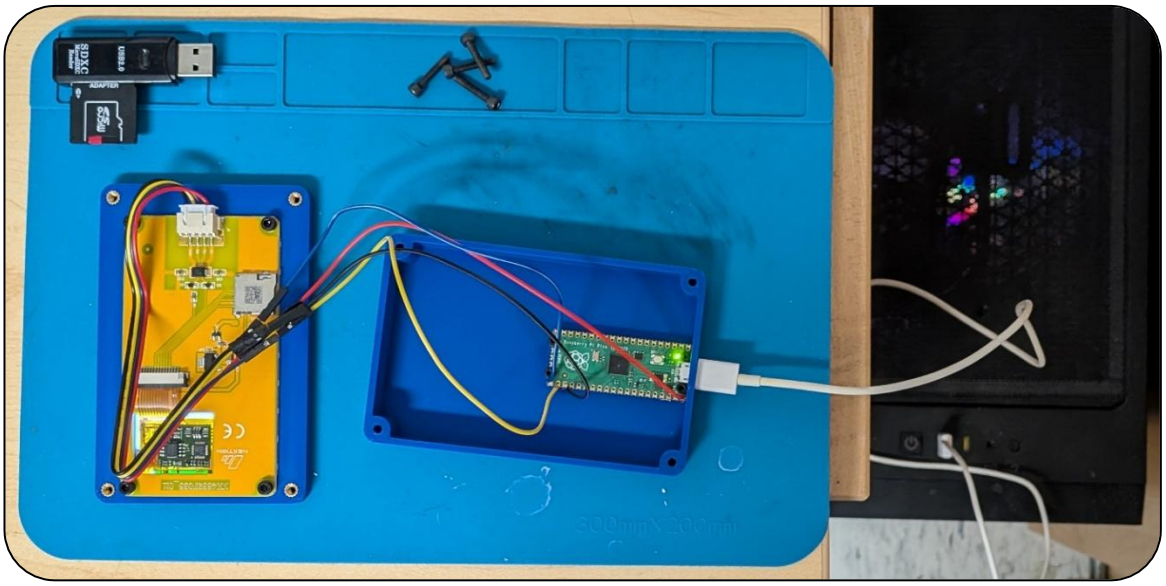
Hardware · Protocol · Code Architecture · AI Analysis Engine · Threat Simulator

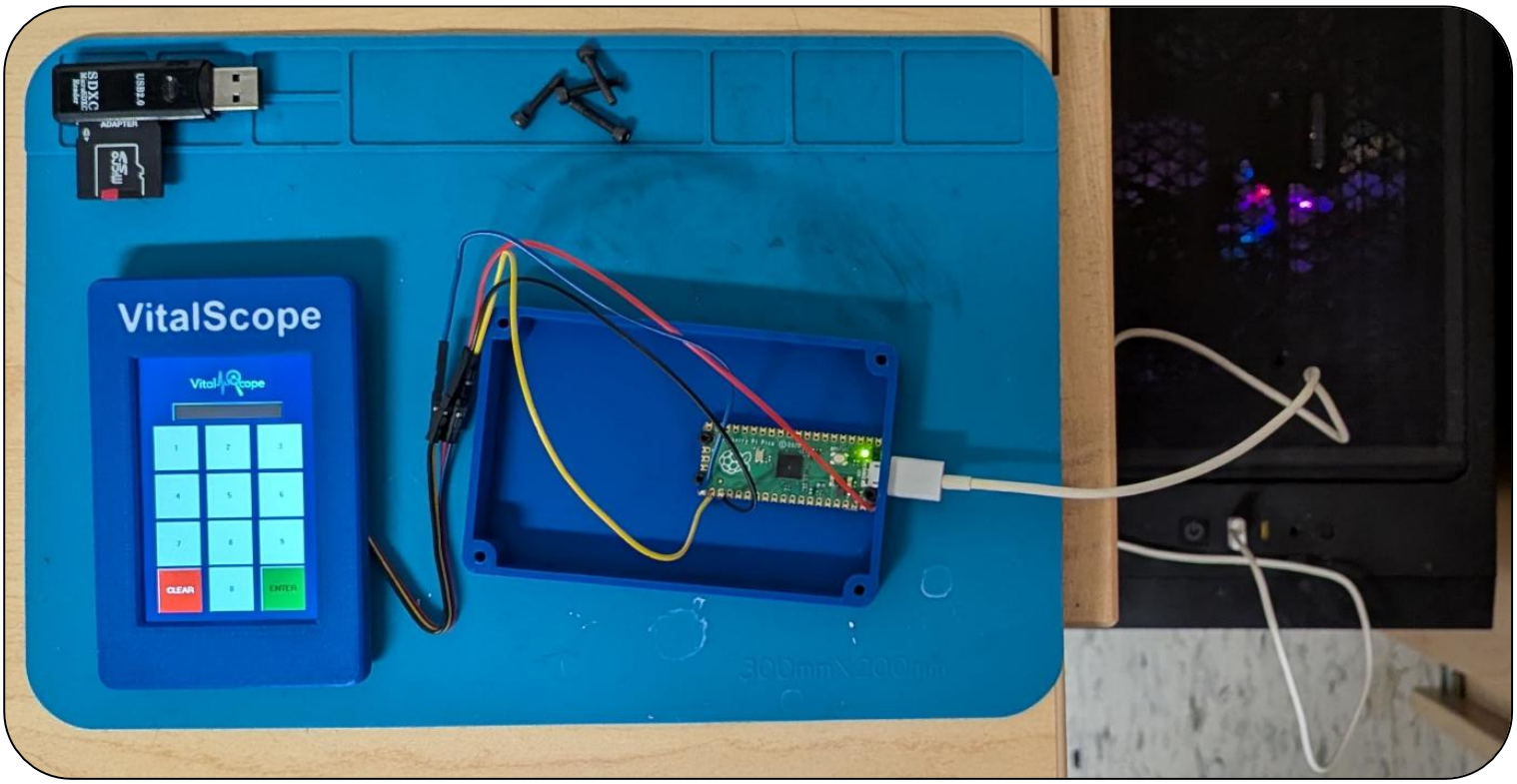
1. System Overview

VitalScope is a PC-to-display telemetry bridge that surfaces live system metrics, network diagnostics, and security data on a Nextion touchscreen. Three hardware components form the signal chain:

Component	Role
Windows PC	Runs the Nextion Control Station Python application; performs all data collection (CPU, RAM, disk, network, security scans, AI analysis).
Raspberry Pi Pico	Acts as a USB CDC serial bridge, translating plain-text commands from the PC into the Nextion UART protocol and forwarding Nextion print events back to the PC.
Nextion Enhanced Display	Renders pages, handles touch input, and emits page-change events back through the chain via its print() function.

NOTE: The PC application never speaks directly to the Nextion. All communication flows through the Pico, which handles protocol translation in both directions.




VitalScope
CLEAR
ENTER

VitalScope
CLEAR
ENTER

1.1 File Structure

File	Responsibility
`main.py`	Entry point — creates the Tk root window and launches NextionControlStation.
`app.py`	Main application class — UI layout, serial connection management, listener thread, page routing, and the Threat Simulator panel.
`pages.py`	PageHandlersMixin — per-page data refresh methods: dashboard, network, portscan, procs (NEW), lock. Also network quality loop and dashboard live loop.
`ai_analysis.py`	NEW — Pure-Python deterministic security analysis engine. No external dependencies or API keys. Scores processes, ports, users, and firewall state; yields a streaming report.
`network_utils.py`	Pure network helpers — quality scoring, speed test, DNS lookup, ARP device count, port check. No GUI or serial dependencies.
`security_scan.py`	Windows security scanner — process enumeration, user accounts, firewall state, listening ports. SecurityScanWindow GUI now includes an AI Analysis tab.
`ui_widgets.py`	Reusable Tkinter factories — styled_btn, styled_entry, section_frame, ElementCard.
`config.py`	All constants — baud rate, delays, RGB565 colours, GUI palette, fonts, PAGE_REFRESH_MAP (now includes 'procs').
`pico_bridge.py`	MicroPython firmware running on the Pico — UART bridge, page name detection, boot sequence. Now recognises b'page_procs'.

2. Hardware Layout

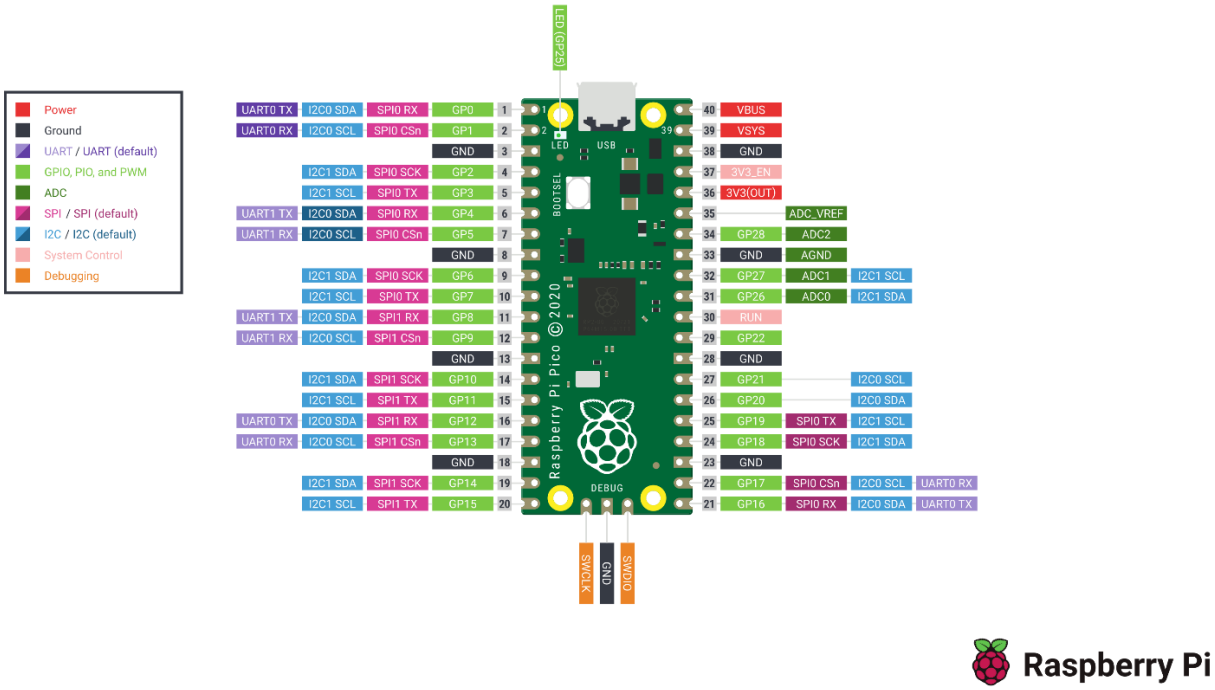
2.1 Signal Chain

```
PC  ──(USB CDC / virtual COM port)──▶  Pico  ──(UART0 @ 9600 baud)──▶  Nextion

Nextion  ──(UART0 TX)──▶  Pico  ──(USB CDC stdout)──▶  PC Listener Thread
```

2.2 GPIO Pin Assignment

GPIO Pin	Direction	Connected To	Purpose
GPIO 16	TX (out)	Nextion RX	Sends commands to the display
GPIO 17	RX (in)	Nextion TX	Receives print() output from display
USB (built-in)	Bidirectional	PC USB port	CDC serial — commands in, events out
Onboard LED	Output	N/A	Lit solid while bridge is running



2.3 UART Configuration

Parameter	Value
UART bus	UART0
Baud rate	9600
TX pin	GPIO 16
RX pin	GPIO 17
Command terminator	0xFF 0xFF 0xFF (3-byte Nextion requirement)
Boot delay	5 seconds (BOOT_DELAY_S — allows Nextion startup to complete)

3. Communication Protocol

3.1 Three-Layer Stack

Layer	Medium	Format
PC → Pico	USB CDC (virtual COM port)	Newline-delimited ASCII, one command per line
Pico → Nextion	Hardware UART0 @ 9600 baud	ASCII command + 0xFF 0xFF 0xFF terminator
Nextion → Pico → PC	UART RX → USB CDC stdout	ASCII print() strings, forwarded by Pico

3.2 PC → Pico (USB CDC)

Commands are sent one per line, terminated with \n. The Pico reads from sys.stdin using select() for non-blocking I/O. Example commands:

```
t_ip.txt="192.168.1.10"      # update text element
g_cpu.val=72                  # set gauge/numeric value
pc_status.pco=11648           # set foreground colour (NX_GREEN)
```

3.3 Pico → Nextion (UART)

The Nextion requires every command terminated with exactly three 0xFF bytes. send_to_nextion() appends this automatically. Common Nextion command forms:

Command Form	Example	Effect
`name.txt="value"`	`t_ip.txt="10.0.0.1"`	Set a text element's string content
`name.val=N`	`g_cpu.val=72`	Set a numeric or gauge value (0–100)
`name.pco=N`	`g_cpu.pco=11648`	Set foreground colour (RGB565 integer)
`name.bco=N`	`t0.bco=63488`	Set background colour (RGB565 integer)
`page NAME`	`page main`	Navigate to a named Nextion page

3.4 Nextion → Pico → PC (Events)

When the user changes pages, the Nextion page-load event calls print() to emit the page name over its UART TX pin. The Pico's persistent RX buffer accumulates bytes and dispatches complete 0xFF 0xFF 0xFF-terminated packets to handle_nextion_message(). Recognised page names:

```
PAGE_NAMES = [
    b'page_dashboard',
    b'page_network',
    b'page_portscan',
    b'page_main',
    b'page_lock',
    b'page_procs',   # NEW
]
```

The portscan page additionally emits a port number after the page name in the same UART chunk. The Pico detects both tokens and prints portscan_init <N> to the PC, which stores the value for use ~0.5 s later when the page refresh fires.

3.5 Nextion RGB565 Colour Constants

Constant	RGB565 Value	Colour	Usage
NX_GREEN	11648	Bright green	Connected / OK status
NX_RED	63488	Bright red	Error / disconnected status
NX_YELLOW	65504	Bright yellow	Warning / fair quality
NX_GREY	33840	Mid grey	Inactive / placeholder text
NX_WHITE	65535	White	Default text colour
NX_BLACK	0	Black	Background / off state

4. Code Architecture

4.1 Page Refresh Flow

Step	Where	Action
1	Nextion HMI	Page-load event executes print("page_<n>")
2	Pico — handle_nextion_message()	Detects page name in raw UART bytes, prints to USB stdout
3	PC — listener thread	Reads line; if page_* prefix, immediately stops loops for the old page synchronously (not via Tkinter queue)
4	PC — root.after(0, _handle_incoming)	Schedules dispatch on main thread; calls _on_page_change()
5	PC — _on_page_change()	Updates active_page_var, looks up PAGE_REFRESH_MAP, checks enabled checkbox
6	PC — _run_page_refresh()	Daemon thread: waits 0.5 s settle delay, then calls mapped method(s)
7	PC — page handler method	Collects data, calls self.send() for each Nextion command
8	PC — self.send() / _send_raw_cmd()	Acquires _serial_lock, writes command + \n to COM port

4.2 Page → Refresh Method Map

Defined in config.py as PAGE_REFRESH_MAP. All methods implemented in pages.py (PageHandlersMixin):

Page Name	Method	What It Sends / Does
main	`page_dashboard_refresh()`	Identical to dashboard — Pico boots to main page.
dashboard	`page_dashboard_refresh()`	CPU %, RAM %, disk %, IP, hostname, pc_status / pi_status. Starts 3-second live loop.
network	`page_network_refresh()`	Internet status, DNS, device count, download/upload speed. Starts network quality monitor loop.
portscan	`page_portscan_refresh()`	Runs scan_firewall(), scores ports by PORT_RISK_SCORES, writes top 7 to label + score element pairs.
procs	`page_procs_refresh()`	NEW — Runs scan_processes(), scores each unknown proc via ai_analysis._score_process(), writes top 4 to name/score/flag element triplets.
lock	`page_lock_refresh()`	Flashes t0 background red/black three times.

4.3 Background Loop Management

Loop	Page(s)	Interval	Stop mechanism
Dashboard live loop (_dash_loop)	dashboard, main	3 seconds	_dash_stop_event — set by _stop_dash_loop() called from listener thread on page change
Network quality loop (_net_quality_loop)	network	PING_INTERVAL (3 s default)	_net_stop_event — set by _stop_net_loop() called from listener thread on page change

THREAD SAFETY: Loop stop calls happen synchronously on the listener thread, not via root.after(). This prevents stale writes from a previous page's loop arriving after the new page has loaded.

4.4 Serial Thread Safety

Mechanism	What it protects	Detail
threading.Lock (_serial_lock)	All serial.write() calls	_send_raw_cmd() acquires the lock before writing and releases after flush(). Concurrent threads queue rather than interleave bytes.
root.after(0, fn)	All Tkinter widget mutations	Background threads never touch widgets directly. Updates are scheduled via root.after(0, ...) or _queue_log().

5. AI Analysis Engine — ai_analysis.py

Ai: ai_analysis.py provides deterministic, rule-based security analysis with no external dependencies and no API keys.

The engine receives the dict returned by security_scan.run_full_scan() and yields the analysis report line-by-line as a Python generator. The caller (SecurityScanWindow._run_ai_analysis) consumes each line and appends it to the GUI text widget with a short sleep, producing a streaming effect.

5.1 Entry Point

```
from ai_analysis import analyse

# scan_results is the dict from run_full_scan()
for line in analyse(scan_results):
    append_to_widget(line)
    time.sleep(0.02)  # streaming effect
```

5.2 Scoring — Processes

_score_process(proc) evaluates a single flagged process and returns (score: int, findings: list[str]). Score range is 0–100. The function is also imported directly by pages.py for use in the procs page refresh.

Signal	Max Points	Detail
Name entropy (very high ≥ 4.2)	30	Shannon entropy of the filename stem — measures character randomness
Name entropy (elevated ≥ 3.5)	15	Elevated but below 'very random' threshold
Suspicious path location	20 per finding	Running from \temp\, \AppData\Local\Temp\, \Downloads\, \Desktop\, etc.
Path unresolvable	20	Process path cannot be read — may be hidden or SYSTEM-level
Non-standard path root	20	Not under C:\Windows\, C:\Program Files\, or C:\Program Files (x86)\
Extreme memory (≥ 1.4 GB)	25	Flagged as extreme — possible data staging or crypto-mining
High memory (≥ 488 MB)	15	Above normal for an unknown process
Elevated memory (≥ 97 MB)	5	Moderately elevated
Long name (> 25 chars)	10	Unusually long process name
Very short name (≤ 2 chars)	15	Suspiciously short stem
Consonant run ≥ 5 chars	+points	Long run of consonants with no vowels (e.g. 'xvzqtbmf')
Digit ratio > 40%	+points	Name is primarily digits — unusual for a real program
Hex string match	+points	Full filename stem matches [0-9a-f]{6,}

5.3 Scoring — Ports

_port_analysis(listeners) evaluates individual high-risk ports and dangerous port combinations:

Check	Detail
Individual high-risk ports	Each port in HIGH_RISK_PORTS (excluding BENIGN_PORTS like 135) generates a finding naming the port, service, and bound process.
Risky port combinations	Specific port pairs are flagged as more dangerous together than individually (e.g., SMB+NetBIOS, RDP+SSH, MSSQL+MySQL, FTP+Telnet, RDP+SMB).
Simultaneous risk count	If ≥ 5 high-risk ports are open at once, an attack-surface warning is generated.

5.4 Scoring — Users & Firewall

Category	Check	Finding Condition
Users	Admin count	More than 2 administrator accounts active
Users	Never-logged-in accounts	Account has 'Never' as last login but is still active
Users	Stale admin accounts	Inactive account that holds admin privileges
Firewall	Profile disabled	Any firewall profile (Domain / Private / Public) has state OFF
Firewall	Inbound rule count	Fewer than 5 inbound rules — firewall may not be meaningfully configured
Firewall	Outbound rule count	Fewer than 5 outbound rules — no egress filtering

5.5 Overall Risk Score

_overall_risk_score() produces a composite 0–100 score from four weighted components:

Component	Max Points	Calculation
Processes	50	Mean of the top-3 individual process scores, scaled by 0.5
Ports	25	5 points per port finding, capped at 25
Users	15	5 points per user finding, capped at 15
Firewall	10	5 points per firewall finding, capped at 10

Score Range	Label
0–24	CLEAN
25–49	LOW RISK
50–74	MODERATE RISK
75–100	HIGH RISK

5.6 Memory Outlier Detection

_outlier_memory_processes() identifies processes whose memory usage is a statistical outlier across the flagged set. The threshold is mean + 2 standard deviations. At least 3 flagged processes are required for outlier detection to activate. Outliers are reported in their own report section and flagged in the recommendations.

5.7 Report Structure

The analyse() generator yields a structured plain-text report with the following sections, in order:

- Header block — timestamp, overall rating, score
- Executive Summary — high-level counts and key findings
- Process Analysis — deep-dive on processes scoring ≥ 50, with PID, memory, path, and individual findings
- Memory Outliers — statistical outlier processes (if any)
- Port & Network Analysis — individual port risks and dangerous combinations
- User Account Analysis — admin count, unused accounts, stale admin accounts
- Firewall Analysis — per-profile state and rule count concerns
- Recommendations — prioritised, actionable steps with specific commands where applicable

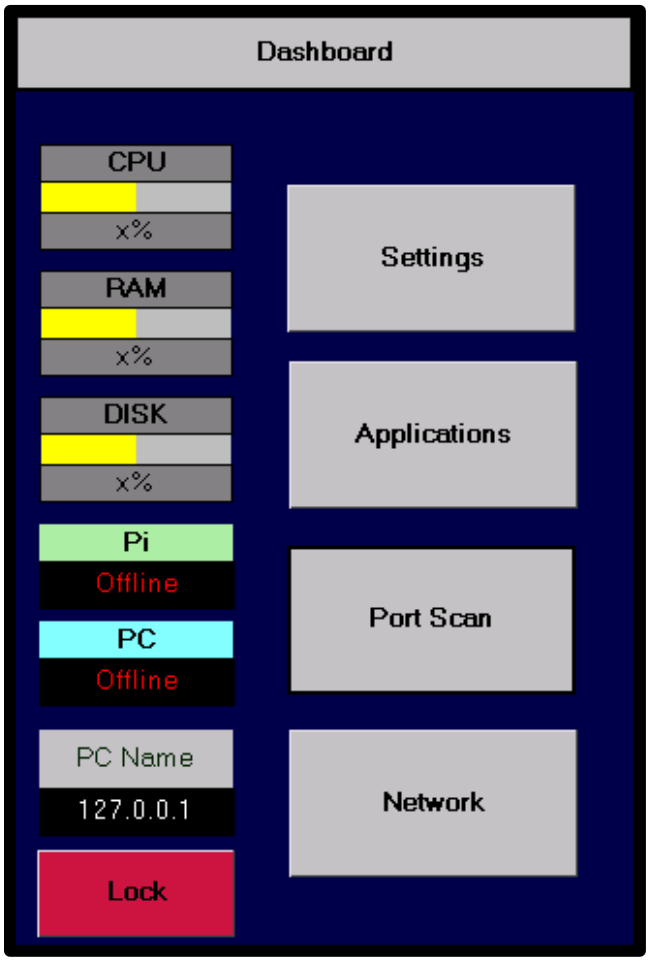
Processes scoring below 50 are suppressed from the detailed section to reduce noise. Processes scoring below 20 are noted in the summary as 'no specific concerns beyond whitelist absence.'

5.8 Constants Reference

Constant	Value	Purpose
ENTROPY_SUSPICIOUS	3.5	Shannon entropy threshold — elevated character randomness
ENTROPY_VERY_RANDOM	4.2	Shannon entropy threshold — almost certainly machine-generated name
MEM_HIGH_KB	500 000	~488 MB — flagged as high memory for an unknown process
MEM_EXTREME_KB	1 500 000	~1.4 GB — flagged as extreme
RISK_LOW	25	Overall score threshold — LOW RISK rating
RISK_MODERATE	50	Overall score threshold — MODERATE RISK rating
RISK_HIGH	75	Overall score threshold — HIGH RISK rating

6. Page Handlers — pages.py

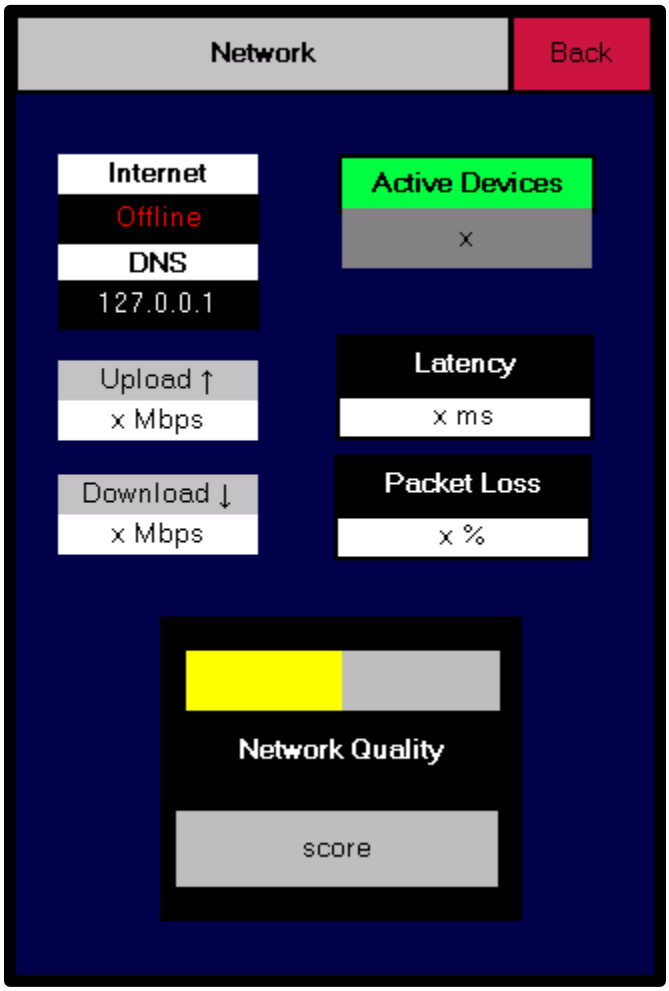
6.1 Page: Dashboard



page_dashboard_refresh() pushes all static data then starts the 3-second live CPU/RAM/Disk loop:

- CPU %, gauge value, gauge colour (g_cpu.val / g_cpu.pco)
- RAM %, gauge value, gauge colour (g_ram.val / g_ram.pco)
- Disk %, gauge value, gauge colour (g_disk.val / g_disk.pco)
- PC hostname (t_pcname.txt), IP address (t_ip.txt)
- pc_status.txt / pc_status.pco, pi_status.txt / pi_status.pco
- Starts _dash_loop — refreshes CPU, RAM, disk every 3 seconds

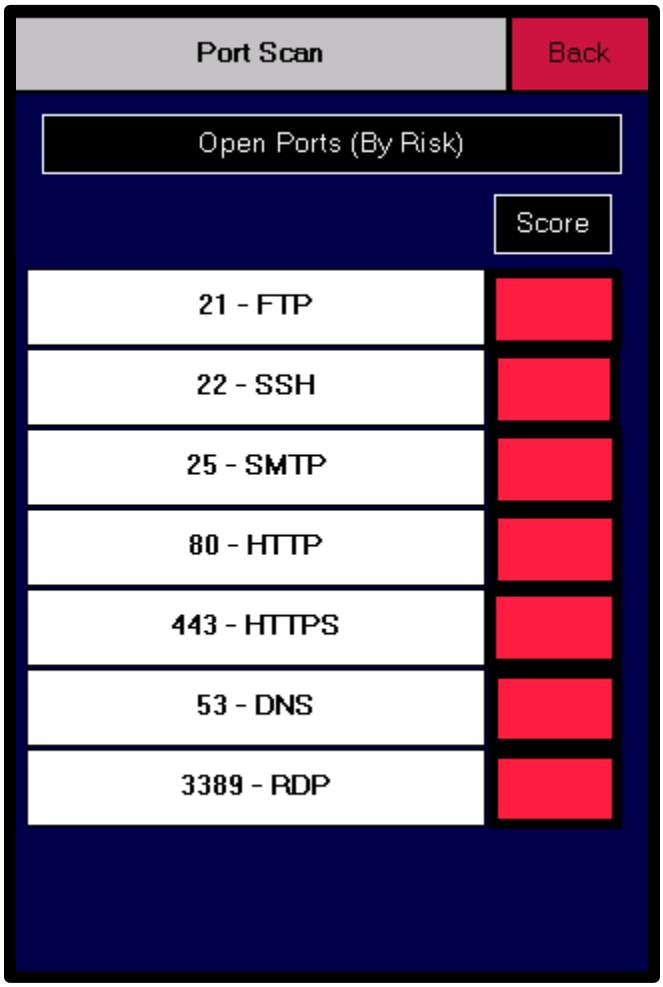
6.2 Page: Network



page_network_refresh() spawns a background thread (_network_full_refresh) that runs five sequential steps:

Step	Element(s) Updated	Detail
1. Internet check	int_status.txt, int_status.pco	TCP probe to 8.8.8.8:53; green = online, red = offline
2. DNS lookup	t_DNS.txt	First DNS server from ipconfig /all, or 'N/A'
3. LAN device count	t_dev.txt	Dynamic ARP entries (arp -a)
4. Speed test	dwnld_status.txt, upld_status.txt	Downloads up to 4 MB from a test URL; upload estimated as 30% of download. Aborts if user leaves page.
5. Quality loop	j0.val, j0.pco, t5.txt, t3.txt, t7.txt	Starts _net_quality_loop — 4 TCP probes every PING_INTERVAL seconds, updates score, latency, and loss %

6.3 Page: Portscan



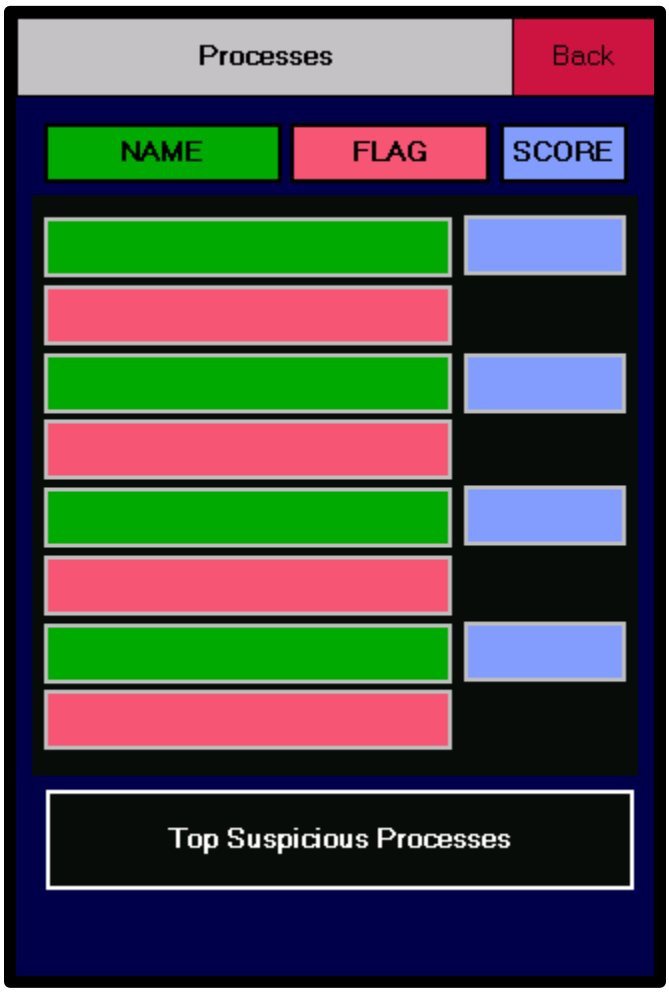
_run_portscan() runs scan_firewall(), scores every listening port, sorts highest risk first, and writes the top 7 to two parallel element lists:

Element list	Nextion elements	Content
PORTSCAN_ELEMENTS	t2, t5, t6, t7, t8, t14, t16	"PORT process_name" (process truncated to 14 chars + ~ if longer)
PORTSCAN_SCORE_ELEMENTS	t21, t22, t25, t80, t443, t53, t3389	Bare risk score integer (0–100)

Port Risk Score Table

Port	Service	Risk Score
23	Telnet	95
445	SMB	92
3389	RDP	90
21	FTP	85
1433	MSSQL	82
25	SMTP	80
3306	MySQL	80
135	RPC	75
5985	WinRM HTTP	72
22	SSH	70
5986	WinRM HTTPS	68
53	DNS	60
8080	HTTP alt	50
80	HTTP	45
8443	HTTPS alt	35
443	HTTPS	20
(any other)	—	30 (default)

6.4 Page: Procs



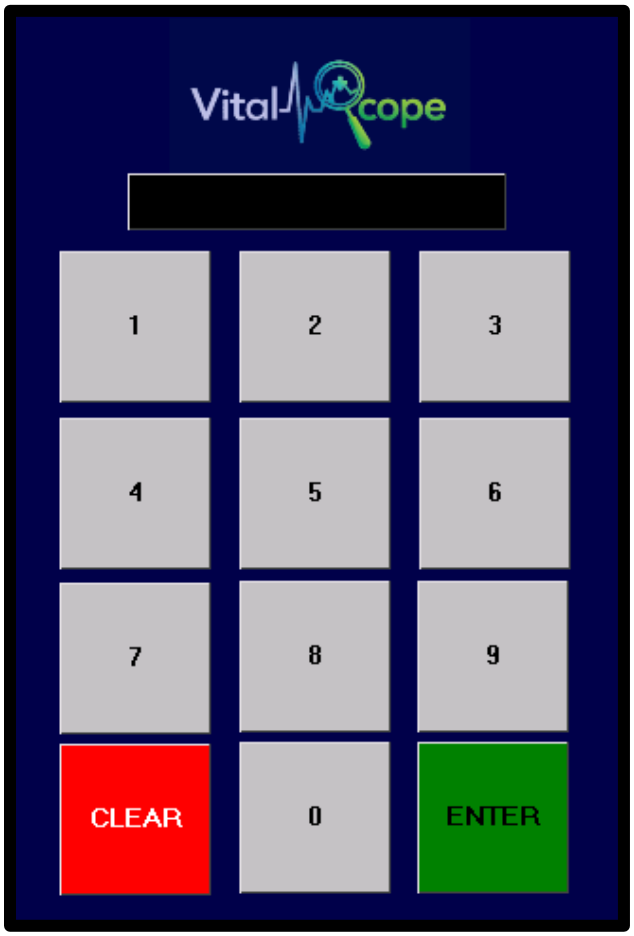
page_procs_refresh() spawns _run_procs() in a background thread:

- Calls scan_processes() to enumerate non-whitelisted running processes.
- Calls _score_process() from ai_analysis.py for each flagged process.
- Sorts by score descending and writes top 4 to three parallel element lists.

Element list	Nextion elements	Content
PROCS_NAME_ELEMENTS	t2, t9, t15, t13	Full process name (no truncation)
PROCS_SCORE_ELEMENTS	t6, t10, t16, t12	Score as "N/100"
PROCS_FLAG_ELEMENTS	t7, t8, t14, t11	First finding reason (category prefix stripped)

Unused rows in all three lists are cleared. If no unknown processes are found, t2 is set to "All clear" and all other elements are cleared.

6.5 Page: Lock



page_lock_refresh() flashes t0 background red/black three times (0.3 s per state). Aborts early if the user navigates away from the lock page mid-animation.

7. Security Scan GUI — security_scan.py

7.1 Tabs

Tab	Content
PROCESSES	Treeview of non-whitelisted processes. Columns: name, PID, memory, flag, path. Sortable by memory and flag. Right-click to open file location or copy path.
USERS	Treeview of all local accounts with last-login timestamp, admin status, and active state. Admin accounts highlighted yellow; inactive accounts dimmed.
FIREWALL & NETWORK	Grouped treeview: firewall profile states, inbound/outbound rule counts, network adapters, and all LISTENING ports with bound process and risk indicator.
AI ANALYSIS	NEW — Streaming text widget that shows the ai_analysis report line-by-line. Auto-runs after every scan. Can also be triggered independently via [RUN ANALYSIS].

7.2 AI Analysis Tab

Workflow:

- _start_ai_analysis() is called automatically from _display_results() after each scan.
- It switches to the AI Analysis tab and kicks off _run_ai_analysis() in a daemon thread.
- _run_ai_analysis() consumes the analyse() generator, sleeping 20 ms between lines to create a streaming effect.
- As lines arrive, _append_ai_line() colour-tags them on the main thread (red for HIGH, yellow for MODERATE / warnings, green for CLEAN, dim for separators).
- On completion, _ai_analysis_done() resets UI state and optionally pushes the summary to the Nextion via _push_ai_to_nextion().

7.3 Scan Functions

scan_processes()

Uses tasklist /FO CSV /NH and cross-references psutil for full executable paths. Any process not in KNOWN_SAFE_PROCESSES is flagged.

```
{
  "flagged": [{"name": str, "pid": int, "mem_kb": int, "path": str|None}],
  "total": int,
  "error": str|None
}
```

scan_users()

Uses net user, net user <n>, and net localgroup Administrators.

```
{
  "accounts": [{"name": str, "last_login": str, "is_admin": bool, "active": bool}],
  "error": str|None
}
```

scan_firewall()

Uses netsh advfirewall, ipconfig /all, netstat -ano, and tasklist.

```
{
  "profiles":  {"Domain": {"state": "ON"|"OFF"}, ...},
  "rules":     {"inbound": int, "outbound": int},
  "adapters":  [{"name": str, "ip": str, "mac": str}, ...],
  "listeners": [{"port": int, "proto": str, "pid": int, "process": str}, ...],
  "error":     str|None
}
```

7.4 Known-Safe Process Whitelist

KNOWN_SAFE_PROCESSES is a frozenset of ~70 lowercase process names. Any process name not in this set is returned in the 'flagged' list. The whitelist includes core Windows system processes, common browsers, productivity applications, and dev/serial tools used with this project (thonny.exe, putty.exe, wireshark.exe, etc.). Add additional entries to reduce false positives for your specific environment.

8. Threat Simulator — app.py

The panel contains two buttons:

Button	Scenario	Pages affected	What it injects
MALWARE SIM	Simulates a malware infection	page_procs	Navigates to page_procs, then writes 4 fake high-risk process entries to the name, score, and flag element triplets. Uses realistic-looking malicious names: svch0st.exe (92/100), xvzqtbmf.exe (85/100), winlogon32.exe (71/100), a3f9b2c1.exe (63/100).
DDoS SIM	Simulates a DDoS attack in progress	page_network	Navigates to page_network, then collapses quality score to 4, sets latency to 850 ms, packet loss to 75%, internet status to 'Degraded' (red), download to 0.1 Mbps, upload to 0.0 Mbps.

9. Networking Tools — network_utils.py

All functions are pure (no GUI or serial dependencies) and safe to call from background threads.

Function	Returns	Description
get_network_quality()	(score, avg_ms, loss_pct)	4 TCP probes to PING_HOST:53; score 0–100, avg latency ms, packet loss %
check_internet()	bool	TCP probe to 8.8.8.8:53 with 3 s timeout — fast online/offline check
get_dns_servers()	str	First DNS server IP from ipconfig /all; Windows-only; returns 'N/A' on failure
get_speed_mbps(stop_flag)	(dl_mbps, ul_mbps)	Streams up to 4 MB from test URLs. stop_flag aborts mid-download. Upload = 30% of download.
count_local_devices()	int	Dynamic ARP entries from arp -a — counts recently-seen LAN devices
check_port(host, port, timeout)	bool	TCP connection attempt; True = port open

Network Quality Scoring

Round-trip time	Points per probe (max 4 probes)
< 20 ms	25
< 50 ms	20
< 100 ms	15
< 200 ms	10
≥ 200 ms	5
Timeout / failure	0

10. Pico Bridge Reference — pico_bridge.py

10.1 Boot Sequence

Step	Action
1	Wait BOOT_DELAY_S (5 s) for the Nextion to finish its own startup.
2	Send 'page main' to navigate to the main dashboard page.
3	Set pi_status.txt = "Connected" and pi_status.pco = 11648 (NX_GREEN).
4	Set pi_con.val = 1 to signal Pico connectivity to the display.
5	Print 'page_main' to stdout so the PC listener knows which page is active.

10.2 Main Loop

PC → Nextion

Reads newline-terminated commands from sys.stdin using select() for non-blocking I/O. Commands starting with DASHBOARD_PREFIXES (pi_, cpu_, ram_, disk_) trigger a 'page main' switch first. The command is then forwarded with the 3-byte terminator and echoed back as [PC_CMD] <cmd>.

Nextion → PC

Content found in raw bytes	Action
Recognised PAGE_NAMES entry	Prints page name to stdout (e.g. page_procs); calls on_page_load() for Pico-side actions.
Anything else	Printed as [NEX_MSG] <text> for debug visibility.

10.3 nextion_get(component)

Queries a Nextion element's value. Sends 'get <component>', waits up to 500 ms for a 0x70 response packet (format: 0x70 <ASCII> 0xFF 0xFF 0xFF). Returns the decoded string, or None on timeout.

10.4 on_page_load(page_name)

Pico-side actions fired immediately on page detection — no PC round-trip required. Currently sets pi_status on the dashboard page only.

11. Configuration Reference — config.py

11.1 Serial / Timing

Constant	Default	Description
DEFAULT_BAUD	9600	Baud rate for USB CDC (PC↔Pico) and UART0 (Pico↔Nextion)
DEFAULT_DELAY	0.12 s	Wait between consecutive serial.write() calls
BOOT_DELAY_S	5 s	Pico waits this long after power-on before sending to Nextion

11.2 Network Quality

Constant	Default	Description
PING_HOST	"8.8.8.8"	Target host for TCP latency probes (Google DNS port 53)
PING_INTERVAL	3.0 s	Seconds between network quality updates on the network page
QUALITY_GAUGE	"j0"	Nextion progress-bar element for quality score

11.3 PAGE_REFRESH_MAP

```
PAGE_REFRESH_MAP = {
    "main":      ["page_dashboard_refresh"],
    "dashboard": ["page_dashboard_refresh"],
    "network":   ["page_network_refresh"],
    "portscan":  ["page_portscan_refresh"],
    "procs":     ["page_procs_refresh"],   # NEW
    "lock":      ["page_lock_refresh"],
}
```

To add a new page: (1) add its entry here, (2) implement page_<n>_refresh() in pages.py, (3) add b'page_<n>' to PAGE_NAMES in pico_bridge.py.

11.4 GUI Colour Palette

Variable	Hex	Role
BG / BG2 / BG3	#0d0f12 / #13161b / #1a1e26	Window, panel, widget backgrounds (darkest to lightest)
BORDER	#252a35	Subtle border lines between sections
ACCENT	#00e5ff	Primary highlight (cyan) — headings, focus rings, send buttons
ACCENT2	#ff6b35	Secondary highlight (orange) — page names, badges
GREEN / RED / YELLOW	#39ff14 / #ff3131 / #ffd600	Status colours — connected, error, warning

Variable	Hex	Role
TEXT / TEXT_DIM	#c8d0e0 / #4a5568	Normal and de-emphasised text

12. Adding a New Page — Checklist

Step	File	Action
1	pico_bridge.py	Add b"page_<n>" to PAGE_NAMES list.
2	config.py	Add "<n>": ["page_<n>_refresh"] to PAGE_REFRESH_MAP.
3	pages.py	Implement def page_<n>_refresh(self): in PageHandlersMixin.
4 (if needed)	app.py	If the page needs a persistent background loop, add _stop_xxx_loop / _start_xxx_loop helpers and a threading.Event, following the dashboard or network pattern.
5	Nextion HMI	Add a page-load event that calls print("page_<n>"). No other Nextion-editor changes are required — all logic lives in Python.

13. Known Constraints & Notes

Area	Constraint / Note
Platform	network_utils.get_dns_servers() and all security_scan functions require Windows (ipconfig, tasklist, netstat, net user, netsh).
AI engine	ai_analysis.py is fully deterministic — no ML model, no API, no randomness. Results are reproducible for the same input. The 'AI' in the name refers to analyst-style heuristic reasoning, not a neural network.
Speed test upload	Upload speed is estimated as 30% of measured download. No real uplink test is performed.
ARP device count	count_local_devices() only counts devices appearing in the ARP cache (recent communication). May undercount on quiet networks.
Process whitelist	KNOWN_SAFE_PROCESSES is static. Newly installed applications appear as 'unknown' until added manually.
Procs page — 4-row limit	The current Nextion procs page layout has 4 display rows. Only the top 4 highest-scored unknown processes are shown.
Portscan page — 7-row limit	The current portscan layout has 7 rows. Only the top 7 highest-risk listening ports are shown.
Nextion buffer	The Pico accumulates bytes in a persistent bytearray across loop iterations, dispatching only on complete 0xFF 0xFF 0xFF-terminated packets. This prevents split-packet issues for multi-token output like page_portscan + port_137.
Tkinter thread safety	All widget updates from background threads must use root.after(0, ...). Direct widget mutation from non-main threads causes silent failures on Windows.
Serial write delay	DEFAULT_DELAY (120 ms) is conservative. Reduce cautiously — the Nextion can drop rapid writes on slower displays.
Threat Simulator	Simulation data is not cleared automatically on page navigation. Press [CLEAR SIM] or navigate to any page to restore live values via the normal refresh flow.